No of additional sheets 2

LOURDES MATHA COLLEGE OF SCIENCE & TECHNOLOGY

LOURDES HILLS, KUTTICHAL P. O., THIRUVANANTHAPURAMM - 695 574

21/30

ANSWER BOOK

Name Anoop M.S Class No. 11 Branch MCA

Subject Design and Analysis of Algorithms Semester S3

Mark Tabulation

Mark Divisions	Q. 1	Q. 2	Q. 3	Q. 4	Q. 5	Q. 6	Q. 7	Q. 8	Q. 9	Q. 10	Q. 11	Q. 12	Q. 13	Q. 14	Q. 15
Total															
Grand Total															

1) Backtracking

* it is a part of constraint satisfaction problem
* it is one of the problem solving / algorithm design strategy
* it is an algorithm technique to solve an problem by an incremental way
* it uses recursive approach to solve the problem
* we can say that backtracking is used to find all possible combinations to solve an optimization problem
* backtracking the problem can be categorized into 3 categories

1. Decision problem: Here we find whether there is any feasible solutions
2. Optimization problem: Here we find whether there exist any best solution.

...ES MATHA COLLE... ...ill LS, KUTTICHAL P. O., THIRU... ...NSWER BOOK

* Enumeration problem: Here we find all possible feasible

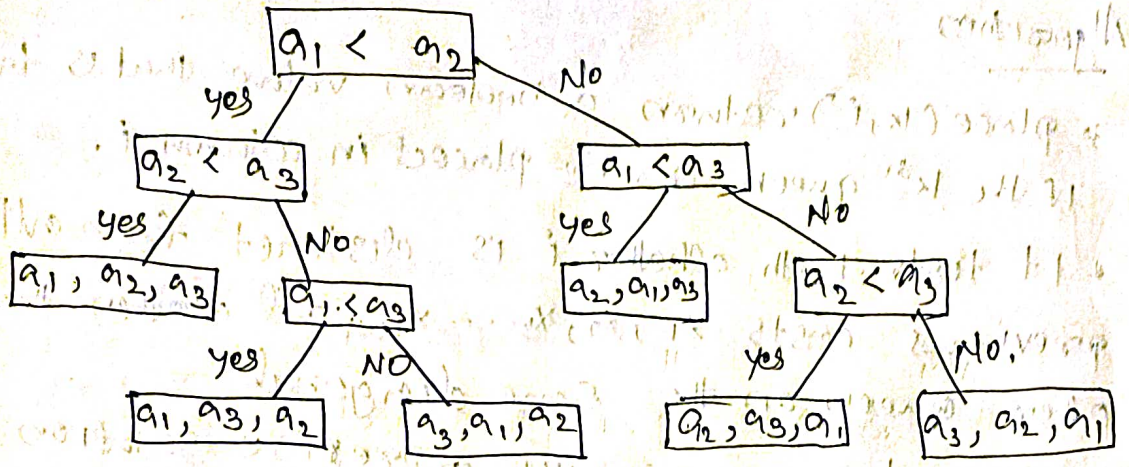
Branch and Bound

* optimization problem
* It is one of the problem solving strategy
* solution is represented in state space like backtra
* It is useful to solving optimization problem.. only minimization not max
* It involves a bounding function
* In branch and bound have a set of solution. then find the bounding function. the algorithm will mak decision to find optimal solution.

3, Decision tree

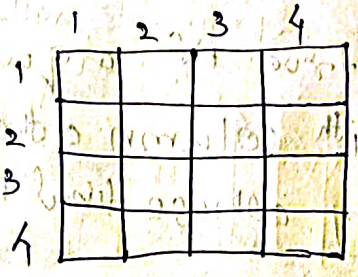
* It is a full binary tree.
* It shows the comparision between elements that a executed by an appropriate sorting algorithm op on an input of a given size.
* control clutch movement and all other conditio the algorithm are ignored
* In a decision tree there will be an ordery of
* So total leaves will be n!. ie, total number comparision.
* If tree height is h then surely $n! \leq 2^h$ (tree will be
* Taking an example of comparing a_1, a_2 and a_3
* let Subtree will be true condition. ie, $a_i \leq a_j$
* Decision & Right Subtree will be false condi

that is



7). N-Queens problem

* In this problem is to place n-queens in such a manner on a n×n chess board. that no Queens attack each other by. being the same row, column, or diagonal
* for n=1 then problem has a trivial solution and no. solution exist for n=2, and n=3
* If n=4 then we have n×n board i.e 4×4 size. and 16C4 Solutions
* based on the condition we create a tree called state space tree.



4×4

* Since we have to place 4 queens such that q_1, q_2, q_3, q_4 on the chess board
* such that no two queens attack each other.
* each queen must be placed on a different row i.e, we put queen i on row i

Algorithm

* place(k,i) return a boolean value that is to if the kth queen can be placed in column i.
* It tests both whether i is distinct from all previous costs $x_1, x_2, x_3 \cdots x_{k-1}$ and whether ther other Queen on the same diagonal
* using place, we give a prerise solution n-queens problem

```
1* Place (k,i)
2* {
3* for j ← 1 to k-1
4* do if (x[j] = i)
5*  or (Abs x[j] - i) = (Abs (j-k))
6* then return false;
7* return true;
8* }
```

* Place (k,i) return true if an queen can be pl in the kth row and i^{th} column otherwise return
* x[] is a global array whose final k-1 values hav set
* Abs (r) returns the absolute value of r.

```
1* N-Queens (k,n)
2* {
* for i ← 1 to n
* do if place (k,i) then
```

```
5. {
6.   X[k] ← i;
7.   if (k==n) then
8.   write (x[1...n]);
9.   else
10.  N-Queens (k+1, n);
11. }
12. }
```

8) Travelling salesman problem (TSP)

Given a set of cities and distance between every pair of cities, the problem is to find the shortest possible route that visits every city exactly once and returns to the starting point

Problem

- let $G(V, E)$ be a directed graph with edge cost
- the variable c_{ij} is defined such that $c_{ij} > 0$ for all i and j and $c_{ij} = \infty$ if $\langle i, j \rangle \notin E$
- let $|V| = n$ and assume $n > 1$
- A tour of G is a directed simple cycle that includes every vertex in V

* The cost of a tour is the sum of the cost of edges on the tour

* The travelling salesperson problem is to find tour of minimum cost

Solution

* A tour - A simple path that starts and ends at vertex 1.

* every tour consist of an edge $\langle 1, k \rangle$ for some and a path from vertex k to 1.

the path from vertex k to vertex 1 goes through vertex in $V - \{1, k\}$ exactly once.

If the tour is optimal, then the path from k to 1 going through all vertices in $V - \{1, k\}$, Hence the of optimality holds

let $g(i, S)$ be the length of a shortest path starting at vertex i going through all vertices in S, terminating at vertex 1.

the function $g(1, V - \{1\})$ is the length of an optimal salesperson tour.

From the principle of optimality it follows that

$$g(1, V - \{1\}) = \min_{2 \le k \le n} \{ c_{1k} + g(k, V - \{1, k\}) \}$$

Generalizing (1) we get for $(i \notin S)$

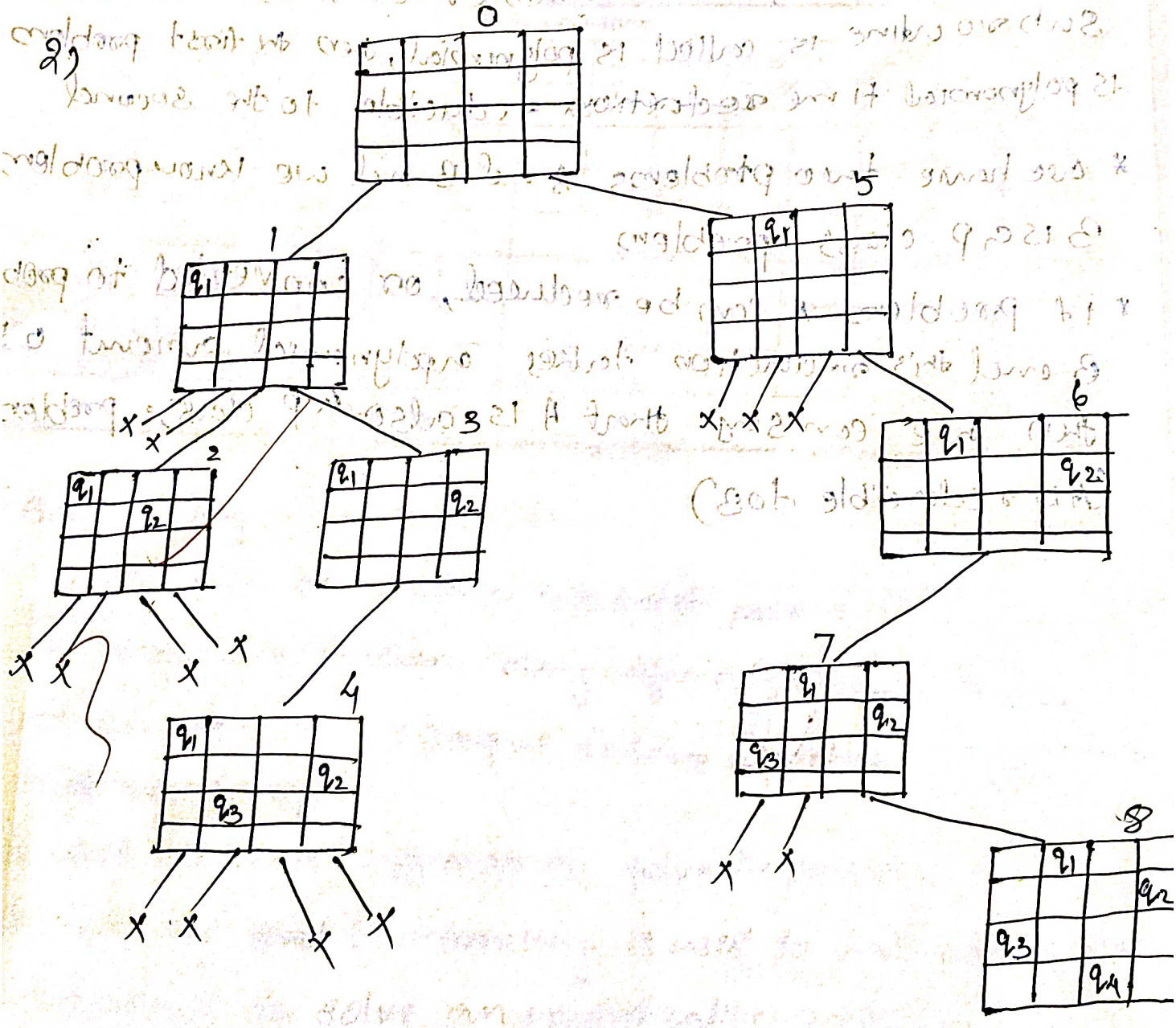
$$g(i, S) = \min_{j \in S} \{ c_{ij} + g(j, S - \{j\}) \}$$

* $g(i, V-\{i\})$ can be solved if we know $g(k, V-\{k\})$ for all choices of k

* The g values can be obtained by using (2)

$$g(i, \phi) = c_{i1}, 1 \le i \le n$$

2)



Polynomial time reduction

* In computational complexity theory a polynomial reduction is a method for solving one problem using another
* If both time required to transform the first problem to the second, and the number of times the second subroutine is called is polynomial, then the first problem is polynomial time ~~reducible~~ reducible to the second
* We have two problems A and B and we know problem B is NP class problem
* If problem A can be reduced, or converted to B and this reduction takes polynomial amount of time then we can say that A is also NP class problem (A is reducible to B)

